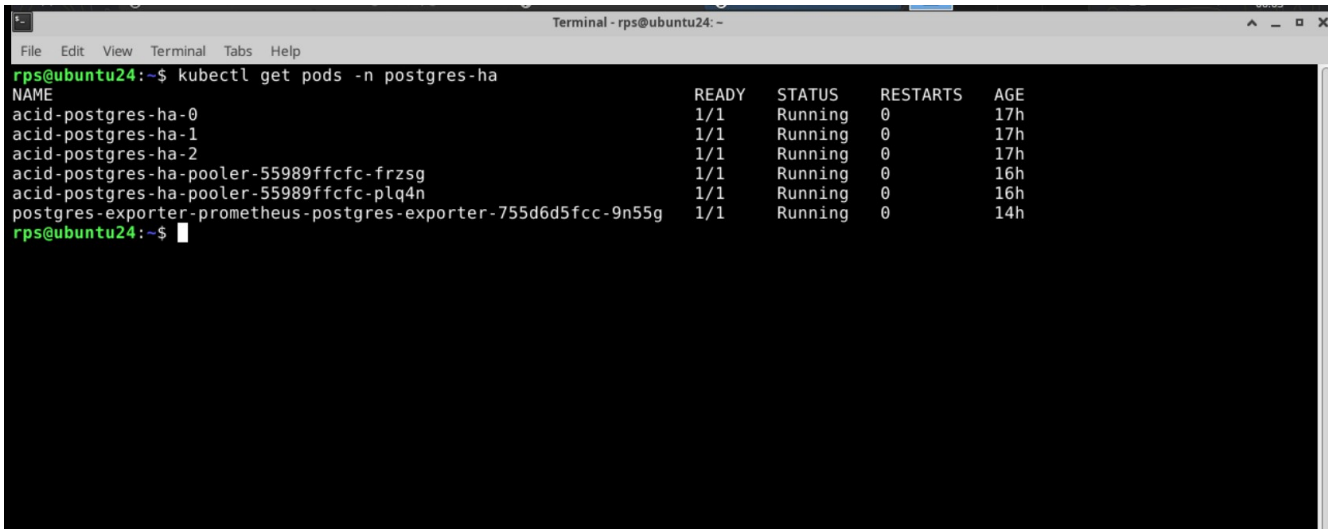


PostgreSQL Backup & Recovery Lab Manual

pgBackRest - Full, Differential, Incremental, Restore and PITR

Target environment: Kubernetes namespace postgres-ha with a 3-node PostgreSQL HA cluster.



```

Terminal - rps@ubuntu24: ~
File Edit View Terminal Tabs Help
rps@ubuntu24:~$ kubectl get pods -n postgres-ha
NAME                                READY   STATUS    RESTARTS   AGE
acid-postgres-ha-0                  1/1     Running   0           17h
acid-postgres-ha-1                  1/1     Running   0           17h
acid-postgres-ha-2                  1/1     Running   0           17h
acid-postgres-ha-pooler-55989ffcfc-frzsg  1/1     Running   0           16h
acid-postgres-ha-pooler-55989ffcfc-plq4n  1/1     Running   0           16h
postgres-exporter-prometheus-postgres-exporter-755d6d5fcc-9n55g  1/1     Running   0           14h
rps@ubuntu24:~$

```

Figure 1. Existing lab environment supplied for this manual.

Item	Value
Namespace	postgres-ha
PostgreSQL pods	acid-postgres-ha-0, acid-postgres-ha-1, acid-postgres-ha-2
Pooler	acid-postgres-ha-pooler-*
Exporter	postgres-exporter-prometheus-postgres-exporter-*
Stanza used in this manual	acid-postgres-ha

WARNING: This manual is intentionally restore-safe. Backups are taken from the HA cluster, but destructive restore and PITR exercises are performed in an isolated recovery target. Do not overwrite the live HA PGDATA during training.

Table of Contents

1. Lab objectives and learning outcomes
2. Architecture and pgBackRest concepts
3. Prerequisites and safety rules
4. Lab 0 - Inspect the Kubernetes HA cluster
5. Lab 1 - Configure pgBackRest
6. Lab 2 - Full backup
7. Lab 3 - Incremental backup
8. Lab 4 - Differential backup
9. Lab 5 - Restore a database cluster
10. Lab 6 - Point-in-time recovery (PITR)
11. Lab 7 - Verification, retention and housekeeping
12. Troubleshooting guide
13. Instructor validation checklist
14. Command quick reference
15. References

1. Lab Objectives and Learning Outcomes

This lab teaches physical PostgreSQL backup and recovery with pgBackRest in a Kubernetes-based HA environment. The exercises focus on operational DBA skills rather than only command syntax.

Objectives

- Understand the relationship among PostgreSQL data files, WAL, backups, and PITR.
- Configure a pgBackRest stanza and repository.
- Enable and verify continuous WAL archiving.
- Create full, incremental, and differential backups.
- Inspect backup metadata and understand restore dependencies.
- Restore a complete PostgreSQL cluster into an isolated recovery target.
- Recover to a timestamp before an accidental change using PITR.
- Validate recovery and identify common Kubernetes/pgBackRest failures.

Expected duration

Lab	Approx. time
Lab 0 - Environment inspection	20 min
Lab 1 - Configuration	40 min
Lab 2 - Full backup	25 min
Lab 3 - Incremental backup	20 min
Lab 4 - Differential backup	20 min
Lab 5 - Restore	45 min
Lab 6 - PITR	60 min
Lab 7 - Validation and retention	25 min

2. Architecture and pgBackRest Concepts

pgBackRest performs physical backups of the PostgreSQL cluster and manages archived WAL. The repository may be a shared POSIX filesystem, NFS/RWX persistent volume, or supported object storage such as S3. All database members that execute archive-push must be able to reach the same repository.

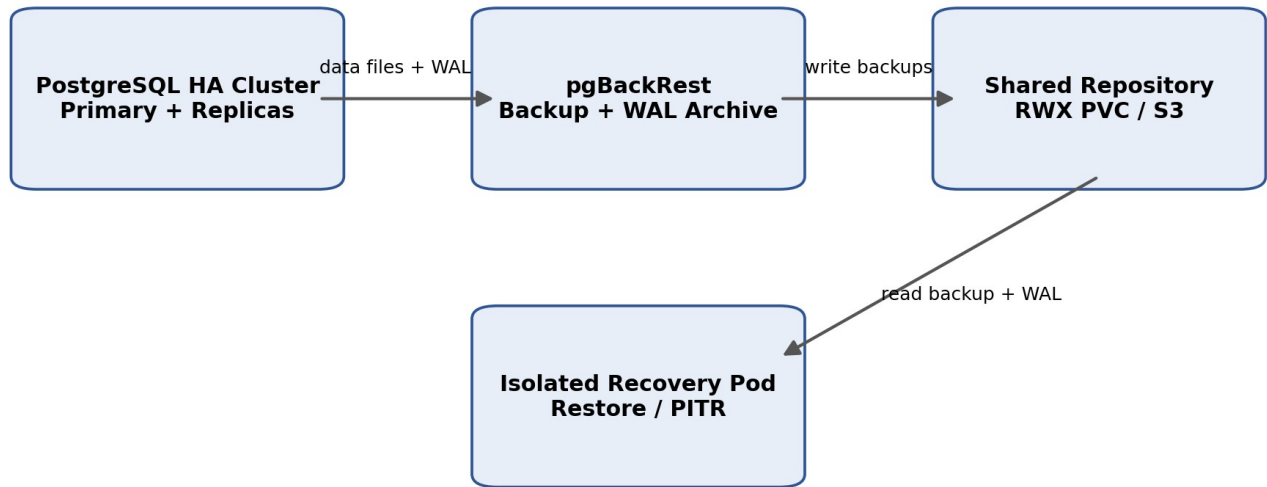
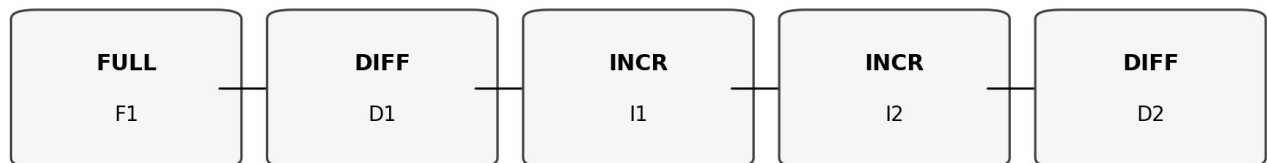


Figure 2. Recommended training architecture.

2.1 Backup types

Type	What is copied	Dependency	Typical use
Full	All required cluster files	None	Baseline and periodic recovery anchor
Differential	Files changed since the last full backup	Last full	Faster than full; simpler chain than incremental
Incremental	Files changed since the last successful backup	Previous backup chain	Smallest/faster routine backup when change volume is low



Restore dependency: DIFF depends on last FULL; INCR depends on the previous backup chain.

Figure 3. Backup-chain relationship.

2.2 Why WAL is required for PITR

A physical backup gives a consistent starting point. WAL records contain the changes that occurred after that point. During recovery, PostgreSQL restores the base backup and replays archived WAL until the chosen recovery target. If required WAL is missing, PITR cannot continue to that target.

2.3 pgBackRest terminology

Term	Meaning
Stanza	A named configuration that describes one PostgreSQL cluster and its repository relationship.
Repository	Storage location for backup sets, manifests, metadata, and archived WAL.
archive-push	Command PostgreSQL calls to send a completed WAL segment to pgBackRest.
archive-get	Command used during recovery to retrieve WAL from the repository.
Backup label	Unique identifier for a backup set; shown by pgbackrest info.
PITR	Restore a backup and replay WAL only up to a target time/LSN/XID or other supported target.

3. Prerequisites and Safety Rules

3.1 Required software and access

- kubectl access to namespace postgres-ha.
- Administrative access to the PostgreSQL cluster for lab-only configuration changes.
- pgBackRest installed in the PostgreSQL runtime image, sidecar, or supported operator integration.
- A repository reachable from every PostgreSQL member that can perform WAL archiving.
- Enough repository capacity for at least one full backup plus WAL generated during the lab.
- An isolated restore target using the same PostgreSQL major version as the backup source.

WARNING: Installing packages interactively inside a running Kubernetes pod is useful only for short experiments. Changes disappear when the pod is recreated. For a repeatable lab, use a custom image or the PostgreSQL operator's supported extension mechanism.

3.2 Repository requirement for HA

Do not use a different local repository inside each PostgreSQL pod. For an HA cluster, archive-push from whichever node is primary must write to the same durable repository. A shared RWX volume or object-storage repository is appropriate.

3.3 Variables used throughout the lab

```
export NS=postgres-ha
export STANZA=acid-postgres-ha
export POD0=acid-postgres-ha-0
export POD1=acid-postgres-ha-1
export POD2=acid-postgres-ha-2
```

4. Lab 0 - Inspect the Kubernetes HA Cluster

Exercise 0.1 - Confirm pods

```
kubectl get pods -n $NS -o wide
```

Expected: the three PostgreSQL pods are Running and Ready. Pooler and exporter pods may also be present, as in the supplied screenshot.

Exercise 0.2 - Identify the Patroni primary

Try the Patroni REST API from each database pod:

```
for p in $POD0 $POD1 $POD2; do
  echo "==== $p ===="
  kubectl exec -n $NS $p -- sh -c 'curl -s http://127.0.0.1:8008/ 2>/dev/null || true'
done
```

Look for the member whose JSON contains role = primary (some Patroni versions may use leader terminology in other views). If curl is not available, use your existing patronictl workflow or run a SQL read/write test through the primary service.

```
export PRIMARY=<primary-pod-name>
echo $PRIMARY
```

Exercise 0.3 - Discover PostgreSQL version and PGDATA

```
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -Atc "select version();"
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -Atc "show data_directory;"
```

Record the output. Use the returned data_directory as <PGDATA> in configuration examples below.

Exercise 0.4 - Verify pgBackRest

```
kubectl exec -n $NS $PRIMARY -- pgbackrest version
```

NOTE: If this command is not found, do not continue by changing the live pod ad hoc. Build or select a PostgreSQL image that includes pgBackRest, then allow the operator to roll the cluster to that image.

5. Lab 1 - Configure pgBackRest

Goal

Create a stanza, connect it to the PostgreSQL data directory, configure retention, enable WAL archiving, and verify the configuration.

Exercise 1.1 - Prepare the repository

This manual assumes the repository is mounted at `/var/lib/pgbackrest` in every PostgreSQL pod. The storage must be durable and shared. Verify the mount from the current primary:

```
kubectl exec -n $NS $PRIMARY -- sh -c 'df -h /var/lib/pgbackrest; touch
/var/lib/pgbackrest/.write-test && rm /var/lib/pgbackrest/.write-test'
```

NOTE: If `/var/lib/pgbackrest` is not present, first provision and mount a shared repository through your Kubernetes/operator configuration. Do not create a host-local directory and assume it is shared.

Exercise 1.2 - Create pgbackrest.conf

Create this configuration in the immutable image, ConfigMap, Secret, or operator-managed configuration so it exists consistently on all database members. Replace `<PGDATA>` with the value discovered in Lab 0.

```
[global]
repo1-path=/var/lib/pgbackrest
repo1-retention-full=2
repo1-retention-diff=3
start-fast=y
process-max=2
log-level-console=info
```

```
[acid-postgres-ha]
pg1-path=<PGDATA>
```

Common location: `/etc/pgbackrest/pgbackrest.conf` or `/etc/pgbackrest.conf` depending on packaging. Verify which file your image uses:

```
kubectl exec -n $NS $PRIMARY -- sh -c 'pgbackrest --stanza=$STANZA --log-level-console=info
check 2>&1 | head -40'
```

Exercise 1.3 - Configure WAL archiving

Required PostgreSQL settings:

```
archive_mode = on
archive_command = 'pgbackrest --stanza=acid-postgres-ha archive-push %p'
```

In an operator-managed Patroni cluster, make these settings through the operator/Patroni configuration, not by permanently editing `postgresql.conf` inside one pod. A typical Zalando Postgres Operator workflow is:

```
kubectl get postgresql -n $NS
kubectl edit postgresql acid-postgres-ha -n $NS
```

Add or update `archive_mode` and `archive_command` under the CR parameters area supported by your operator, save, and allow the operator to apply/restart members as required. The exact CR field layout depends on the installed operator version.

Validation

```
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "show archive_mode;"
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "show archive_command;"
```

Expected: archive_mode = on and archive_command invokes pgbackrest archive-push.

Exercise 1.4 - Create the stanza

```
kubectl exec -n $NS $PRIMARY -- pgbackrest --stanza=$STANZA stanza-create
```

Exercise 1.5 - Check the stanza

```
kubectl exec -n $NS $PRIMARY -- pgbackrest --stanza=$STANZA check
```

Expected end state: check completes successfully. The check command validates configuration and tests archiving where possible.

Exercise 1.6 - Force a WAL switch and verify archiving

```
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "select pg_switch_wal();"
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "select archived_count,
failed_count, last_archived_wal, last_archived_time from pg_stat_archiver;"
```

Expected: archived_count increases and failed_count should remain stable at zero for a clean lab.

6. Lab 2 - Full Backup

Goal

Create an independent baseline physical backup and verify its metadata.

Exercise 2.1 - Create lab data before the backup

```
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -v ON_ERROR_STOP=1 -c "create
database backup_lab;"
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d backup_lab -v ON_ERROR_STOP=1 -c "create
table backup_demo(id int primary key, note text, created_at timestampz default now()); insert
into backup_demo(id,note) values (1,'before full backup'),(2,'baseline row');"
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d backup_lab -c "table backup_demo;"
```

Exercise 2.2 - Run the full backup

```
kubectl exec -n $NS $PRIMARY -- pgbackrest --stanza=$STANZA --type=full --log-level-console=info
backup
```

Observe the start and stop timestamps and the final successful completion message. A full backup is the recovery anchor for the following differential/incremental exercises.

Exercise 2.3 - Inspect backup information

```
kubectl exec -n $NS $PRIMARY -- pgbackrest --stanza=$STANZA info
```

Record the full backup label. Example format resembles YYYYMMDD-HHMMSS.F.

Exercise 2.4 - Validate repository size

```
kubectl exec -n $NS $PRIMARY -- sh -c 'du -sh /var/lib/pgbackrest/* 2>/dev/null || true'
```

7. Lab 3 - Incremental Backup

Goal

Create a backup that contains changes since the most recent successful backup.

Exercise 3.1 - Generate changes

```
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d backup_lab -c "insert into
backup_demo(id,note) values (3,'after full - before incremental');"
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d backup_lab -c "update backup_demo set
note='baseline row updated' where id=2;"
```

Exercise 3.2 - Run incremental backup

```
kubectl exec -n $NS $PRIMARY -- pgbackrest --stanza=$STANZA --type=incr --log-level-console=info
backup
```

Exercise 3.3 - Compare metadata

```
kubectl exec -n $NS $PRIMARY -- pgbackrest --stanza=$STANZA info
```

The incremental backup label should identify its parent chain. Its repository footprint should generally be smaller than the full backup when only a small amount of data changed.

Knowledge check

- If the preceding backup in the incremental chain is unusable, later incremental backups depending on that chain cannot be restored independently.
- The first backup of a new stanza is effectively a full backup even if a different type is requested.

8. Lab 4 - Differential Backup

Goal

Create a backup containing files changed since the last full backup, regardless of any incremental backups in between.

Exercise 4.1 - Generate additional changes

```
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d backup_lab -c "insert into
backup_demo(id,note) values (4,'after incremental - before differential');"

```

Exercise 4.2 - Run differential backup

```
kubectl exec -n $NS $PRIMARY -- pgbackrest --stanza=$STANZA --type=diff --log-level-console=info
backup
```

Exercise 4.3 - Inspect chain

```
kubectl exec -n $NS $PRIMARY -- pgbackrest --stanza=$STANZA info
```

A differential backup depends on the last full backup, not on the incremental backup you created in Lab 3. This makes the restore dependency chain shorter than a long series of incrementals.

Comparison

Characteristic	Full	Differential	Incremental
Backup content	All files	Changes since last full	Changes since last successful backup
Restore dependency	None	Full + selected diff	Full/diff + required incremental chain
Typical backup size	Largest	Medium	Smallest
Typical restore complexity	Lowest	Medium	Highest with long chain

9. Lab 5 - Restore a Database Cluster

Safety model

WARNING: Do not restore over acid-postgres-ha-0/1/2. Patroni and the operator expect to manage those members. Perform the restore into an isolated recovery pod or separate VM with the same PostgreSQL major version and access to the same pgBackRest repository.

Exercise 5.1 - Create an isolated recovery target

Use your organization's approved image that contains the same PostgreSQL major version plus pgBackRest. The pod below is a template; replace <PG_IMAGE_WITH_PGBACKREST> and <RECOVERY_PVC>.

```
apiVersion: v1
kind: Pod
metadata:
  name: pg-restore-lab
  namespace: postgres-ha
spec:
  containers:
    - name: postgres
      image: <PG_IMAGE_WITH_PGBACKREST>
      command: ["sh", "-c", "sleep infinity"]
      volumeMounts:
        - name: restore-data
          mountPath: /restore/pgdata
        - name: backrest-repo
          mountPath: /var/lib/pgbackrest
  volumes:
    - name: restore-data
      persistentVolumeClaim:
        claimName: <RECOVERY_PVC>
    - name: backrest-repo
      persistentVolumeClaim:
        claimName: <SHARED_BACKREST_REPO_PVC>
```

Apply the manifest after filling the placeholders, then verify:

```
kubectl get pod -n $NS pg-restore-lab
kubectl exec -n $NS pg-restore-lab -- pgbackrest version
```

Exercise 5.2 - Put the stanza configuration on the recovery pod

Create a pgBackRest configuration that points pg1-path to /restore/pgdata and repo1-path to the same shared repository used by the source cluster.

```
[global]
repo1-path=/var/lib/pgbackrest
log-level-console=info

[acid-postgres-ha]
pg1-path=/restore/pgdata
```

Exercise 5.3 - Inspect available backups from the recovery target

```
kubectl exec -n $NS pg-restore-lab -- pgbackrest --stanza=$STANZA info
```

Exercise 5.4 - Restore the latest backup

The restore destination must be empty. Confirm before proceeding:

```
kubectl exec -n $NS pg-restore-lab -- sh -c 'ls -la /restore/pgdata'
kubectl exec -n $NS pg-restore-lab -- sh -c 'rm -rf /restore/pgdata/* /restore/pgdata/.[!]*
/restore/pgdata/..?* 2>/dev/null || true'
kubectl exec -n $NS pg-restore-lab -- pgbackrest --stanza=$STANZA --pg1-path=/restore/pgdata --
log-level-console=info restore
```

NOTE: The rm command is intentionally limited to /restore/pgdata in the disposable recovery target. Never substitute the live HA PGDATA path.

Exercise 5.5 - Start restored PostgreSQL manually

Exact binary paths and UID differ by image. Run as the PostgreSQL operating-system user and use a non-conflicting port such as 55432. Example:

```
kubectl exec -n $NS pg-restore-lab -- sh -c 'echo "port=55432" >>
/restore/pgdata/postgresql.auto.conf'
kubectl exec -n $NS pg-restore-lab -- sh -c 'pg_ctl -D /restore/pgdata -l /tmp/restore-
postgres.log start'
```

If your image requires su/su-exec/gosu, use the image-specific method to run pg_ctl as the database owner.

Exercise 5.6 - Verify restored data

```
kubectl exec -n $NS pg-restore-lab -- psql -p 55432 -U postgres -d backup_lab -c "table
backup_demo;"
```

Rows present depend on which backup was selected as latest. With the sequence in this manual, the latest differential should include rows 1-4.

Exercise 5.7 - Restore a specific backup set

Use pgbackrest info to copy a backup label, stop the recovery server, empty the recovery data directory, and restore that exact set:

```
kubectl exec -n $NS pg-restore-lab -- sh -c 'pg_ctl -D /restore/pgdata stop -m fast'
kubectl exec -n $NS pg-restore-lab -- sh -c 'rm -rf /restore/pgdata/* /restore/pgdata/.[!]*
/restore/pgdata/..?* 2>/dev/null || true'
kubectl exec -n $NS pg-restore-lab -- pgbackrest --stanza=$STANZA --set=<BACKUP_LABEL> --pg1-
path=/restore/pgdata restore
```

10. Lab 6 - Point-in-Time Recovery (PITR)

Scenario

You will create a known good row, record a timestamp, then simulate an accidental delete. PITR will recover the database to a time after the good row existed but before the delete committed.

Exercise 6.1 - Ensure a fresh backup exists

Take a fresh full or differential backup before the incident so the recovery starting point is recent.

```
kubectl exec -n $NS $PRIMARY -- pgbackrest --stanza=$STANZA --type=diff backup
```

Exercise 6.2 - Create a good transaction and record time

```
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d backup_lab -c "insert into
backup_demo(id,note) values (100,'must survive PITR');"
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d backup_lab -Atc "select clock_timestamp();"
```

Write down the timestamp as GOOD_TIME. Wait at least several seconds to make the target easier to distinguish.

Exercise 6.3 - Simulate an accidental destructive change

```
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d backup_lab -c "delete from backup_demo where
id in (1,2,3,4,100);"
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d backup_lab -Atc "select clock_timestamp();"
```

Write down this timestamp as BAD_TIME. Confirm that the rows are gone:

```
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d backup_lab -c "table backup_demo;"
```

Exercise 6.4 - Force WAL switch and confirm archiving

```
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "select pg_switch_wal();"
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "select archived_count,
failed_count, last_archived_wal, last_archived_time from pg_stat_archiver;"
```

Exercise 6.5 - Choose a PITR target

Choose a timestamp after GOOD_TIME and before BAD_TIME. Include timezone explicitly. Example only:

```
export TARGET_TIME='2026-09-25 10:15:30+05:30'
```

NOTE: Do not copy the example time literally. Use the timestamp captured in your own lab. The target must be reachable by archived WAL retained in the repository.

Exercise 6.6 - Stop the isolated recovery server and clean its PGDATA

```
kubectl exec -n $NS pg-restore-lab -- sh -c 'pg_ctl -D /restore/pgdata stop -m fast 2>/dev/null
|| true'
kubectl exec -n $NS pg-restore-lab -- sh -c 'rm -rf /restore/pgdata/* /restore/pgdata/.[!]*
/restore/pgdata/..?* 2>/dev/null || true'
```

Exercise 6.7 - Restore to the target time

```
kubectl exec -n $NS pg-restore-lab -- pgbackrest --stanza=$STANZA --pg1-path=/restore/pgdata --
type=time --target="$TARGET_TIME" --target-action=promote --log-level-console=info restore
```

pgBackRest writes recovery settings needed for PostgreSQL to retrieve archived WAL with archive-get and stop at the requested target. PostgreSQL 12+ uses normal configuration plus recovery.signal rather than recovery.conf.

Exercise 6.8 - Start recovered PostgreSQL

```
kubectl exec -n $NS pg-restore-lab -- sh -c 'echo "port=55432" >>
/restore/pgdata/postgresql.auto.conf'
kubectl exec -n $NS pg-restore-lab -- sh -c 'pg_ctl -D /restore/pgdata -l /tmp/pitr-postgres.log
start'
```

Exercise 6.9 - Validate PITR result

```
kubectl exec -n $NS pg-restore-lab -- psql -p 55432 -U postgres -d backup_lab -c "select * from
backup_demo order by id;"
kubectl exec -n $NS pg-restore-lab -- psql -p 55432 -U postgres -d postgres -c "select
pg_is_in_recovery();"

```

Expected: row 100 exists and the destructive delete is not present. After target-action=promote completes, pg_is_in_recovery() should eventually be false.

PITR timeline

Time	Event	Expected after PITR
T0	Latest usable physical backup	Base state restored
T1	Row 100 inserted	Must exist
TARGET	Chosen between T1 and T2	Recovery stops here
T2	Accidental delete	Must NOT be replayed

11. Lab 7 - Verification, Retention and Housekeeping

Exercise 7.1 - Run a post-backup check

```
kubectl exec -n $NS $PRIMARY -- pgbackrest --stanza=$STANZA check

```

Exercise 7.2 - Review archive statistics

```
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "select archived_count,
failed_count, last_archived_wal, last_archived_time, last_failed_wal, last_failed_time from
pg_stat_archiver;"

```

Exercise 7.3 - Understand retention

This manual used repo1-retention-full=2 and repo1-retention-diff=3. pgBackRest automatically expires backups according to policy after successful backups. WAL required for retained backups is kept according to the repository retention rules.

```
kubectl exec -n $NS $PRIMARY -- pgbackrest --stanza=$STANZA expire --log-level-console=info

```

Exercise 7.4 - Produce a training evidence bundle

Capture these outputs for assessment:

- kubectl get pods -n postgres-ha
- pgbackrest version
- pgbackrest --stanza=acid-postgres-ha check
- pgbackrest --stanza=acid-postgres-ha info
- pg_stat_archiver output
- backup_demo contents after restore
- backup_demo contents after PITR

12. Troubleshooting Guide

Symptom	Likely cause	What to check
pgbackrest: command not found	Image does not include pgBackRest	Use a repeatable custom image/operator integration; do not rely on temporary apt install.
stanza-create fails: path invalid	pg1-path does not match actual PGDATA	SHOW data_directory; verify config mounted in the same container.
archive-push fails	Repository/config/permission/network problem	archive_command, repository mount, ownership, logs, pg_stat_archiver.failed_count.
check says no valid backups	No backup yet or stanza/repository mismatch	Run full backup; verify stanza name and repo path.
restore says destination not empty	Old files remain in restore PGDATA	Stop recovery instance and clean only the isolated restore path.
PITR cannot reach target	Required WAL missing/expired	Check archive retention, WAL archive history, target timezone and selected backup.
Changes lost after pod recreation	Configuration/package changed inside ephemeral filesystem	Move configuration/package into image, ConfigMap/Secret, volume, or operator CR.
Backup runs on replica unexpectedly	Command executed on wrong member	Identify current primary before backup; use HA-aware automation in production.
Permission denied in repository	UID/GID mismatch on shared storage	Check mount ownership, securityContext, fsGroup and repository permissions.

Useful diagnostic commands

```
kubectl describe pod -n $NS $PRIMARY
kubectl logs -n $NS $PRIMARY --tail=200
kubectl exec -n $NS $PRIMARY -- pgbackrest --stanza=$STANZA --log-level-console=detail check
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "show archive_command; show archive_mode;"
kubectl exec -n $NS $PRIMARY -- psql -U postgres -d postgres -c "select * from pg_stat_archiver;"
```

13. Instructor Validation Checklist

Check	Pass criteria	Result
Cluster health	All three database pods Running/Ready	[]
pgBackRest installed	pgbackrest version succeeds	[]
Repository	Durable shared repository writable	[]

Stanza	stanza-create/check successful	[]
WAL archive	archived_count increases, failed_count not increasing	[]
Full backup	Full backup visible in info	[]
Incremental	Incremental visible and chained	[]
Differential	Differential visible and based on full	[]
Restore	Isolated instance starts and expected rows exist	[]
PITR	Good row exists; destructive delete absent	[]

14. Command Quick Reference

Task	Command
Version	pgbackrest version
Create stanza	pgbackrest --stanza=acid-postgres-ha stanza-create
Validate	pgbackrest --stanza=acid-postgres-ha check
Full	pgbackrest --stanza=acid-postgres-ha --type=full backup
Differential	pgbackrest --stanza=acid-postgres-ha --type=diff backup
Incremental	pgbackrest --stanza=acid-postgres-ha --type=incr backup
Information	pgbackrest --stanza=acid-postgres-ha info
Restore latest	pgbackrest --stanza=acid-postgres-ha restore
Restore set	pgbackrest --stanza=acid-postgres-ha --set=<LABEL> restore
PITR time	pgbackrest --stanza=acid-postgres-ha --type=time --target=<TIMESTAMP> --target-action=promote restore
Expire	pgbackrest --stanza=acid-postgres-ha expire

15. References

Official pgBackRest User Guide: <https://pgbackrest.org/user-guide.html>

Official pgBackRest Command Reference: <https://pgbackrest.org/command.html>

PostgreSQL 18 Backup and Restore: <https://www.postgresql.org/docs/18/backup.html>

PostgreSQL continuous archiving and PITR documentation:

<https://www.postgresql.org/docs/18/continuous-archiving.html>

PostgreSQL 12+ recovery configuration note: recovery.conf was removed and recovery settings are handled through normal PostgreSQL configuration plus signal files.

Important production note

This training manual uses direct kubectl commands to make the mechanics visible. In production, automate backup policy, repository credentials, monitoring, restore drills, and failure handling through your operator/platform. A backup is operationally trustworthy only when restore and PITR are tested regularly.